

LonghornSilicon Software

Compiler Co-Design Overview

Date: 2026-05-14
Status: Blocks 1 & 2 of 4 in software reference
Audience: Compiler engineers integrating against the chip
Repository: LonghornSilicon/adaptive-precision-attention
Branch: compiler-integration-prep

Purpose. This document is the orientation map for a compiler team integrating against the LonghornSilicon chip. It points to the bit-accurate reference models, the ISA spec, the verification status of each block, and the open questions we want answered to converge the v0.2 reference. Read this first; then jump to the specific references it links.

Contents

1	What LonghornSilicon Builds	3
2	Block Coverage in the Software Reference	3
3	Where Things Live	3
4	Build and Test (One Line)	4
5	Recommended Reading Order for a Compiler Engineer	4
6	Integration Phases	4
7	Open Questions for the Compiler Team	5
8	Verification Summary	5
9	Next Blocks in the Queue	6

1. What LonghornSilicon Builds

A four-block LLM inference accelerator targeting TSMC 16FFC via the TSMC University Program (tape-out target Q3/Q4 2026):

1. **ACU (Attention Compute Unit)** — decides INT8 vs FP16 per attention tile and runs the matmul. Composed of:
 - Precision Controller — ratio gate on raw scores
 - MAC Array — INT8 and FP16 dispatched matmul
2. **KV Cache Engine** — on-chip SRAM / eDRAM with compression on writes, decompression on reads.
3. **Token Importance Unit** — per-token attention-weight accumulator driving mixed-precision retention.
4. **Memory Hierarchy Controller** — L1 SRAM / L2 eDRAM / off-chip LPDDR5 routing.

For compiler co-design, blocks 1 (and its two sub-components) are the first to be fully specified in software. Blocks 2–4 follow the same template as they land.

2. Block Coverage in the Software Reference

Table 1: Reference-model status as of `compiler-integration-prep`.

Block	C++ class	C API	Python	Status
Precision Controller	<code>lhsi::PrecisionController</code>	<code>lhsi_pc_*</code>	<code>PrecisionController</code>	143/143 vs RTL
MAC Array	<code>lhsi::mac::MacArray</code>	<code>lhsi_mac_*</code>	<code>MacArray</code>	numpy agreed
KV Cache Engine	—	—	—	in design
Token Importance Unit	—	—	—	not started
Memory Hierarchy Controller	—	—	—	not started

Each new block follows the same three-language template; see `docs/new_block_blueprint.md` for the per-block scaffolding pattern.

3. Where Things Live

```
sw/
+-- README.md                                (markdown form of this doc)
+-- reference_model/
    +-- README.md                            (API reference, markdown)
    +-- MAC_ARRAY_DESIGN.md
    +-- Makefile                             (build / test / static / shared /
        clean)
    +-- precision_controller_ref.{hpp,cpp,py}
    +-- test_precision_controller_ref.{cpp,py}
    +-- mac_array_ref.{hpp,cpp,py}
```

```

+-- test_mac_array_ref.{cpp,py}
+-- integration_example.py      (end-to-end attention tile)
+-- example_compiler_use.py     (three sophistication levels)

docs/
+-- isa/precision_controller_isa.pdf (ISA spec)
+-- mac_array_design.pdf            (this doc family)
+-- reference_model_api.pdf         (formal API reference)
+-- sw_overview.pdf                (this document)
+-- new_block_blueprint.md         (template for future blocks)

```

4. Build and Test (One Line)

```
cd sw/reference_model && make test-all
```

Builds all C++ tests, runs them, runs all Python tests, prints a final parity statement. Requires Python 3.10+, NumPy, and a C++17 compiler.

Granular targets:

Target	What it does
test-pc	Precision controller C++ test only
test-mac	MAC array C++ test only
test-py	All Python tests
integration	End-to-end worked example (printed report)
shared	libprecision_controller_ref.so + libmac_array_ref.so
static	.a variants
clean	Remove build artifacts

5. Recommended Reading Order for a Compiler Engineer

1. **This document** (top to bottom; you are almost done).
2. docs/isa/precision_controller_isa.pdf, especially §4.1 (worked lowering example) and §4.2 (compiler binding patterns: MLIR / TVM / ONNX / custom IR).
3. docs/mac_array_design.pdf for the MAC-array design decisions (frozen vs. open).
4. docs/reference_model_api.pdf for the full API reference (C++ class, C ABI, Python).
5. sw/reference_model/integration_example.py — run it, then read it side-by-side. The shape of that script is the shape your codegen should produce.

6. Integration Phases

The interface (ISA + reference models) is stable across all four phases. Only the runtime implementation of the operations changes: Python function call → AXI register write → PCIe MMIO.

Table 2: Four-phase compiler integration plan.

Phase	Timeline	Compiler targets	We provide
0	now	Python + C++ reference models	Models, ISA, tests on this branch
1	when ZCU102/104 arrives	AXI-Lite + AXI-Stream on Zynq UltraScale+	Vivado bitstream + PYNQ driver
2	blocks 3/4/5 land	Same AXI, more blocks visible	Larger Vivado project
3	post-tape-out (2027+)	PCIe-attached custom chip	TSMC 16FFC silicon + Linux driver

7. Open Questions for the Compiler Team

These are in flight; the answers shape the v0.2 reference models.

1. **Compilation granularity.** Does your compiler emit individual matmuls plus a precision-gate op, or does it expect a fused `flash_attention(Q, K, V)` op? Affects whether we add a fused-attention reference operation.
2. **FP16 bit-exactness.** Do you require bit-exact parity with our hardware’s eventual FP16 rounding, or is numerically-close sufficient? See `mac_array_design.pdf` §??.
3. **Async issue + token model.** Do you want non-blocking matmul issue today (we would add it to v0.2 now), or wait until the RTL exists?
4. **Cost-model precision.** Are placeholder cycle counts enough for your scheduler, or do you need post-synthesis numbers?
5. **THRESHOLD as a runtime register.** See ISA spec §9, open question 1.

We will track answers in the design docs and tag each decision as **frozen** or **open** so the compiler team knows what they can build against.

8. Verification Summary

Table 3: Current verification status, both blocks.

Block	Test	Status
Precision Controller	143/143 RTL replay tiles, bit-exact (C++ + Py)	pass
Precision Controller	Stateful = batched = stateless	pass
Precision Controller	extern "C" = C++ class	pass
Precision Controller	Canonical edges (spike = 10 / 11)	pass
MAC Array	INT8 matmul = naive int64 reference	pass
MAC Array	FP16 matmul = naive double reference (within tol)	pass
MAC Array	Edge cases (zero, identity, signed)	pass
MAC Array	extern "C" = C++ class	pass
MAC Array	FP16 round-trip helpers	pass
End-to-end	Integration example routes match ground truth	82/18 vs 82/18

9. Next Blocks in the Queue

In order of priority for the compiler co-design effort:

1. **Token Importance Unit.** Accumulator + classifier; ~3–4 days once the keep/demote/evict threshold policy is decided.
2. **KV Cache Engine.** Gated on the compression-algorithm decision (GEAR / RotateKV / Lexico); ~3–4 weeks once decided.
3. **Memory Hierarchy Controller.** Thin allocator first, refine when L1 / L2 / DRAM behavior matters; ~1 week.

Each follows the same three-language template established by blocks 1 and 2.

This document is the canonical entry point for a compiler engineer working with LonghornSilicon. It is versioned alongside the reference models in `LonghornSilicon/adaptive-precision-attention` and updated as each new block lands in software.